3D OBJECT DETECTION

***Abstract*—Three-dimensional object detection is what lets autonomous vehicles, robots, and AR systems make sense of the world around them. Over the past decade or so, the field has gone through three broad phases: an early period where detectors relied on a single sensor—either cameras or LiDAR—working in isolation, a second wave that fused multiple sensors into unified bird's-eye-view representations, and an ongoing third wave shaped by foundation models, occupancy reasoning, and language-aligned detectors that can recognize objects they were never explicitly trained on. We review the landscape across six dimensions—sensor modality, data representation, architecture, detection paradigm, learning strategy, and temporal reasoning—covering over 40 methods. Along the way we compare performance on nuScenes and Waymo, look at latency and parameter counts, and dig into the open problems that still keep these systems from reliable real-world deployment: sensor degradation, covariate shift, labeling bottlenecks, and the challenge of building truly holistic perception.**

***Index Terms*—3D object detection, autonomous driving, LiDAR, multi-modal fusion, bird's-eye view, transformer, deep learning**

I. Introduction

Think about what it takes for a car to drive itself. Every second, its sensors—cameras, LiDAR, radar, ultrasonics—produce somewhere around 40 GB of raw data, and all of it has to be processed in real time just to figure out what's happening outside the window. At the heart of this is 3D object detection: given a stream of sensor readings, figure out where everything is, how big it is, which way it's facing, and how fast it's moving [2]. It sounds straightforward, but unlike 2D detection where you're working with ready-made pixels, 3D detection has to reason about the actual physical world—meters, angles, velocities—from measurements that are sparse, noisy, and full of holes.

The last ten years or so have seen three fairly distinct waves of progress. Early work stuck to a single sensor—either cameras with geometric tricks to recover depth [11] or LiDAR with point-based or voxel-based networks [3], [4]. Around 2022, a second wave showed that fusing cameras and LiDAR into a shared bird's-eye-view (BEV) representation [9], [10], [8] could push accuracy much higher than either sensor alone. More recently, a third wave has started to take shape, driven by occupancy grids that don't need bounding boxes, vision-language models that let detectors recognize objects they weren't trained on, and end-to-end pipelines that fold detection, tracking, and planning into a single network [12].

A. Scope and Contributions

Rather than organizing methods by chronology or by benchmark scores, we look at the field along six dimensions that we think capture the main design choices:

1) **What sensor feeds the detector?** Monocular, stereo, multi-view camera; LiDAR point clouds; multi-modal fusion.
2) **How is raw data represented?** Points, voxels, BEV grids, range views, or neural occupancy fields.
3) **What architecture processes it?** CNNs, transformers, hybrids, or diffusion backbones.
4) **How are objects parametrized?** Fixed anchors, center heatmaps, learned queries, or fully sparse representations.
5) **How is the model trained?** Full supervision, self-supervision, semi-supervision, or language-guided open-vocabulary learning.
6) **Does the model use temporal context?** Single frames, recurrent memory, attention over time, or world-model prediction.

For each axis we identify representative methods, note what worked and what didn't, and point to open questions. Along the way we compare performance on nuScenes and Waymo with latency and parameter counts, and we try to separate problems that are basically solved from ones that still keep researchers up at night.

B. Organization

Section II walks through sensor choices, organizing camera methods by how they handle depth and LiDAR methods by their detection paradigm. Section III covers data representations including the newer occupancy field approaches. Section IV looks at architectures, paying particular attention to attention mechanisms. Section V discusses training strategies from fully supervised to language-guided. Section VI summarizes datasets, metrics, and benchmark results. Section VII digs into the hard problems that remain. Section VIII wraps up.

II. Sensor Modalities

TABLE I
Comparison of sensor modalities for 3D object detection.

Property	Camera	LiDAR	Radar	Fusion
Depth accuracy	Low	High	Medium	High
Semantic richness	High	Low	Low	High
Weather robustness	Low	Medium	High	Med.-High
Cost	Low	High	Medium	High
Range	Medium	Med.-Long	Long	Long

A. Camera-Based Detection

The hardest thing about camera-based detection is obvious: images are 2D, and we need 3D. Different methods deal with this in fundamentally different ways.

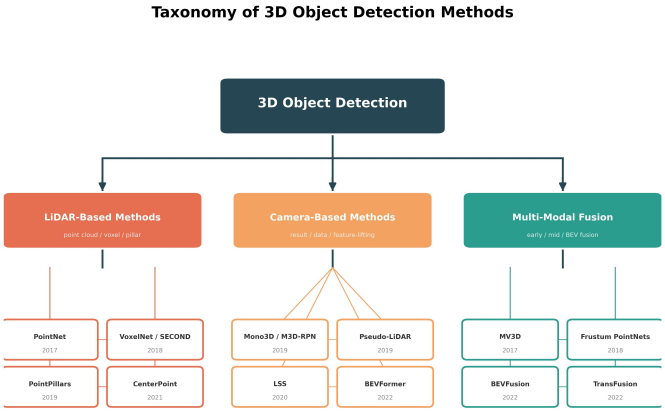

Fig. 1. Taxonomy of 3D object detection methods organized by sensor modality.

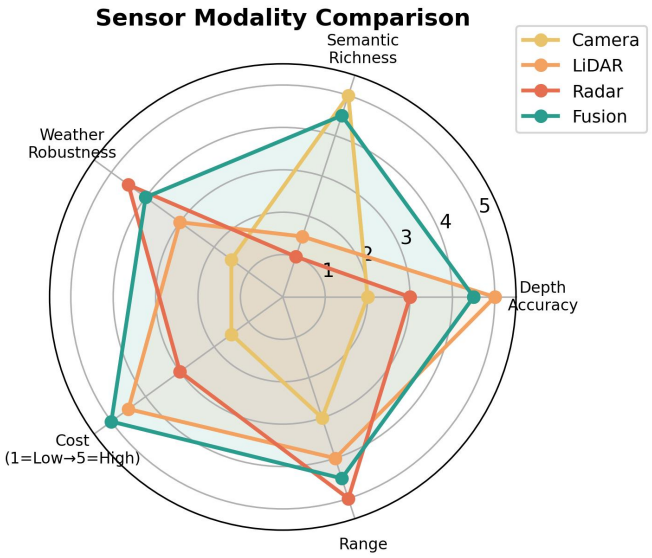

Fig. 2. Radar chart comparing sensor modalities across five key properties.

The simplest approach is **geometric reasoning**—infer 3D boxes directly from 2D detections using known camera geometry and ground-plane constraints. Mono3D does this by scoring candidate 3D boxes against image evidence while enforcing physical plausibility (objects don't float, cars don't intersect). It's fast and doesn't need extra data, but calibration errors hurt badly and you need strong priors about scene layout.

A more popular strategy is **depth estimation**: predict a dense depth map from the image, then lift 2D features into 3D space. CaDDN estimates a categorical depth distribution per pixel and projects into frustum volumes that get pooled into BEV space. BEVDepth jointly optimizes depth estimation with detection and gets noticeably better depth quality as a result. The fundamental problem is that depth uncertainty grows with distance—estimating whether something is 50 or 55 meters away from a single image is genuinely hard [11].

Then there are methods that skip explicit depth entirely.

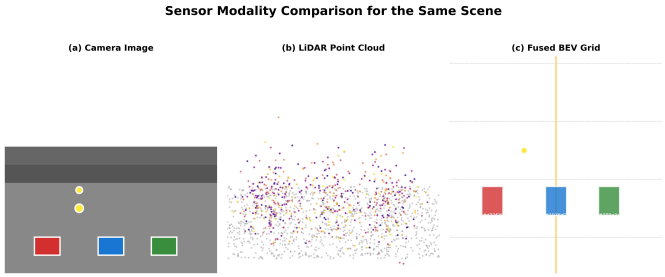

Fig. 3. Comparison of three sensor perspectives for the same driving scene: (a) camera image with rich semantics, (b) LiDAR point cloud with precise geometry, and (c) fused BEV representation.

Attention-based implicit depth models learn to attend to the right 3D locations without ever predicting depth as an intermediate output. DETR3D [13] uses a transformer decoder where object queries attend to multi-view image features; the network figures out depth implicitly during training. PETR encodes 3D position information directly into image features. These avoid the depth discretization errors that plague explicit methods.

The current state of the art for camera-only detection uses **BEV grid transforms**: learned view transformations that project image features onto a top-down grid. BEVFormer [8] is the standout here—each BEV grid cell learns to attend to relevant pixels across camera views and previous time steps through deformable attention. SOLOFusion pushes this further with longer temporal aggregation. These are the best vision-only methods we have, but they still run into the same depth ambiguity problem at long range [11].

B. LiDAR-Based Detection

LiDAR gives you accurate range measurements—each laser pulse bounces off something and tells you exactly how far it is—but the resulting point cloud is sparse, irregular, and unstructured. How you deal with that structure has evolved through a few distinct ideas.

The earliest successful approach was **anchor-based**: define a set of 3D boxes at regular positions and regress offsets. VoxelNet [4] divided space into cubical voxels, encoded the points in each one, and applied 3D convolutions. The problem was that dense 3D convolutions are absurdly wasteful—over 95% of voxels are empty in a driving scene. SECOND [5] solved this by replacing them with submanifold sparse convolutions that only compute at occupied locations, getting roughly a 10× speedup. PointPillars [6] went further by collapsing the vertical dimension entirely, turning voxels into pillars and enabling fast 2D convolutions on the resulting BEV grid. It was a clever trade-off: you lose some fine-grained height information, but you gain real-time inference.

Center-based detection was a genuine rethink. Instead of dense anchors, CenterPoint [7] treats objects as peaks on a BEV heatmap—predict a Gaussian blob at every object center, then regress box attributes from those peak locations.

It eliminated the headache of anchor design and NMS tuning, and it worked better than anything else at the time.

Query-based detectors brought transformers into the picture. TransFusion-L uses learned object queries initialized from heatmap peaks, then refines them through transformer decoder cross-attention. DETR3D does something similar but operates on point clouds directly.

More recently, **fully sparse** detectors argued that even the BEV feature map is unnecessary. FSD [14] (Fully Sparse Detector) never constructs a dense grid at any point—it operates exclusively on non-empty voxel locations from input to output, clustering voxels into objects through a sparse instance recognition module. It achieves competitive accuracy at a fraction of the compute, which matters a lot if you're deploying on a car's embedded computer rather than a server GPU [12].

C. Multi-Modal Fusion

LiDAR gives you precise geometry; cameras give you rich semantics. The obvious idea is to combine them, but *when* you combine them makes a big difference.

Early fusion mixes sensor data before any processing. MV3D projects the LiDAR point cloud onto front and top-down views and fuses those with camera features using region-based pooling. PointPainting is even simpler: run a segmentation model on the image, paint the segmentation scores onto the LiDAR points as extra features, then process everything as a LiDAR point cloud. These approaches are straightforward but brittle—a tiny calibration error or synchronization delay wreaks havoc.

Mid-level (feature) fusion is where most recent work lives. The idea is to process each modality independently through its own backbone, then combine the resulting features. BEVFusion [9] projects both LiDAR voxel features and camera image features into a shared BEV grid through learned transformations and fuses them with convolutions. FusionFormer goes a step further by using a single transformer with shared attention blocks and modality-specific positional encodings, so the two modalities can interact at multiple scales.

Late fusion is the simplest option: detect independently in each modality, then merge the results. CLOCs takes 2D and 3D detection candidates and learns to reinforce detections that appear in both while suppressing ones that don't. It's surprisingly effective considering how simple it is.

The most interesting recent direction is **adaptive fusion**—let the model decide per-location how much to trust each modality. TransFusion [10] does this with transformer cross-attention: LiDAR queries attend to camera features, but the attention can choose to ignore camera features in regions where the image is dark or occluded. DeepFusion generalizes this with learnable uncertainty weights. This matters because in practice, there are plenty of situations where one sensor is unreliable—a camera blinded by the sun, or LiDAR in heavy fog—and the model should know to rely on the other.

1) Emerging Sensors: 4D Imaging Radar: Worth a quick mention: 4D imaging radar has been getting attention as a cheap, weather-proof alternative to LiDAR. Unlike traditional radar that only gives you range and azimuth, 4D radar adds elevation and Doppler velocity, producing something that looks almost like a LiDAR point cloud—at 4–10 cm range resolution—but works in fog, rain, and snow where LiDAR loses over 40% of its detections. RDIoU and RadarNet show that radar-camera fusion can match LiDAR-based performance in clear weather and surpass it when conditions get bad. It feels like this sensor is where LiDAR was five years ago—not yet widely adopted, but the trajectory is clear.

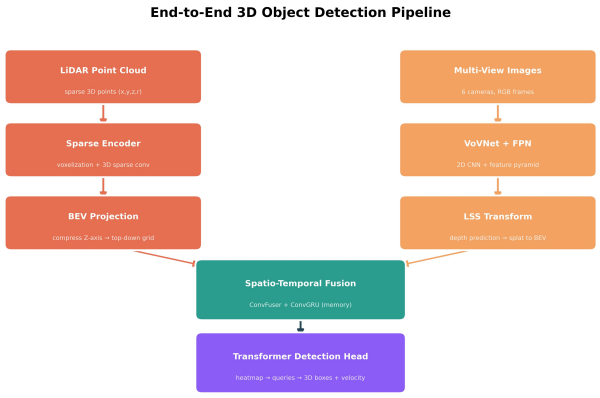

Fig. 4. End-to-end pipeline for 3D object detection: raw LiDAR and camera inputs are processed through parallel feature extractors, projected to BEV space, fused temporally, and decoded by a transformer head.

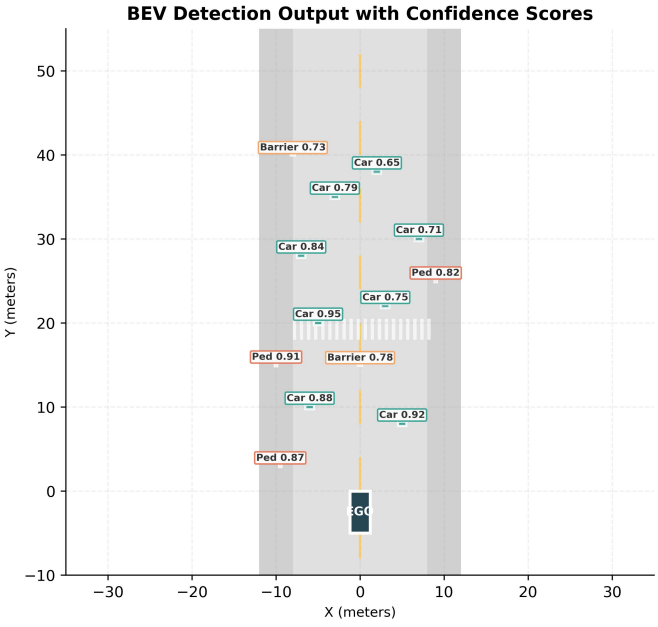

Fig. 5. BEV detection output with colored 3D bounding boxes and confidence scores for cars (teal), pedestrians (red), and barriers (amber) in a road scene.

III. Data Representations

How you represent the raw sensor data is probably the single biggest design decision in a 3D detector. Get this right

and everything downstream is easier. Get it wrong and you're fighting an uphill battle.

A. Point-Based Representations

The most natural representation is just the points themselves. PointNet [3] showed you could process raw LiDAR points directly—each one goes through a shared MLP, then max-pooling aggregates everything into a global descriptor. The big advantage is zero discretization loss: object boundaries stay at native sensor resolution. PointNet++ added hierarchical grouping for multi-scale features. The catch is computational scaling: processing 150k+ points per sweep is expensive, and you usually end up needing aggressive downsampling that throws away information [3]. It's a trade-off that gets worse as LiDAR resolution keeps increasing.

B. Voxel-Based Representations

The alternative is to quantize the space into a regular 3D grid. VoxelNet [4] was the first to do this end-to-end: partition space into cubic voxels, encode the points in each one through a small PointNet, then run 3D convolutions. The problem is that in a typical driving scene, over 95% of voxels are empty, so dense 3D convolutions waste almost all their compute on nothing. SECOND [5] fixed this with submanifold sparse convolutions that only compute at occupied locations, cutting FLOPs by about $10\times$ with negligible accuracy loss. Voxel size is the knob you turn: smaller voxels capture finer geometry but blow up memory quadratically.

C. Projection-Based Representations

The most popular representation by far is the **Bird's-Eye View (BEV)**. The idea is simple: collapse the vertical dimension—through max-pooling or learned aggregation—and you get a neat top-down 2D grid where objects sit at their actual ground-plane positions. BEV preserves metric spatial relationships, objects don't overlap in the same way they do in perspective views, and it plugs naturally into HD maps, trajectory planners, and occupancy grids. Plus it uses efficient 2D convolutions instead of expensive 3D ones. This combination of properties is why BEV became the default representation in modern autonomous driving [8], [9].

Range View (RV) takes a different approach: project the point cloud onto a spherical grid matching the LiDAR's native scanning geometry. It's much more compact—typically 64×2048 cells versus 200×200 for BEV—and maps directly to 2D CNN architectures. RangeDet and RangeNet++ show RV can match BEV accuracy on Waymo with lower latency. The trade-off is spatial distortion: objects far from the sensor cover fewer pixels, objects at different distances have different scales, and bounding box regression becomes noticeably harder [12].

D. Hybrid Representations

Nobody says you have to pick one. PV-RCNN combines voxel efficiency for coarse proposals with point-level refinement for precise localization. Some work blends range view classification with BEV spatial reasoning. These hybrids are usually more expensive to train but often get the best of both worlds.

E. Neural Occupancy Representations

This one is worth paying attention to. Instead of bounding boxes, occupancy networks represent the scene as a continuous volumetric field—every point (x, y, z) maps to an occupancy probability and semantic class through a neural network. OccNet [15] extends this to 4D by adding temporal flow prediction. The real advantage is that occupancy naturally handles things bounding boxes can't: construction barriers, debris, vegetation, anything with irregular shape. This representation has become the standard intermediate output in end-to-end models like UniAD, where it connects perception directly to prediction and planning without the information loss inherent in boxes.

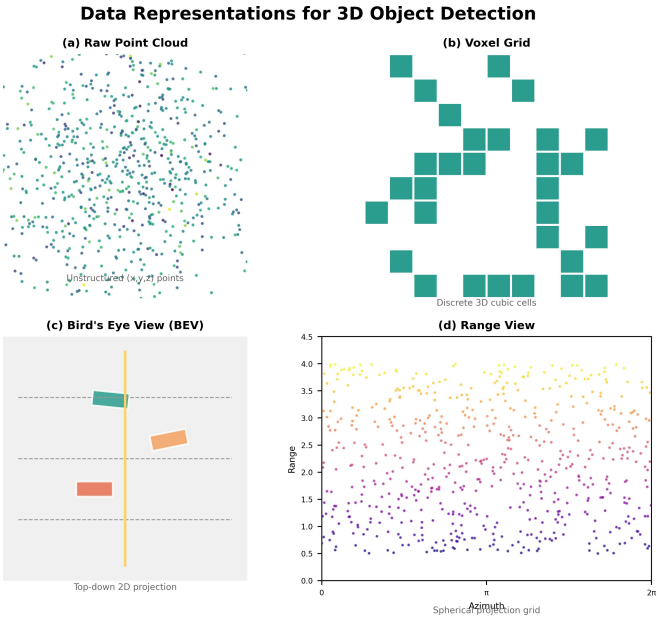

Fig. 6. Common data representations for 3D object detection: (a) raw point cloud, (b) voxel grid, (c) bird's-eye view, and (d) range view projection.

IV. Architectural Paradigms

Given a sensor and a representation, what network architecture actually processes the data? The answer has shifted significantly over the past few years.

A. Convolutional Architectures

CNNs are still everywhere in 3D detection. For images, standard backbones like ResNet, VoVNet, and Swin—often with Feature Pyramid Networks for multi-scale detection—are the default. For LiDAR, submanifold sparse convolutions [5] became the standard because they only compute at occupied voxel locations, cutting FLOPs by over 90% compared to dense 3D convolutions. CenterPoint [7] showed that a well-designed CNN with a convolutional heatmap head can still be competitive with more exotic architectures when paired with the right detection paradigm.

B. Transformer Architectures

What CNNs struggle with is long-range context—a convolution kernel only sees a small local neighborhood. Transformers fix this through self-attention. BEVFormer [8] uses spatiotemporal cross-attention where BEV grid queries attend to multi-view camera features and previous-frame features. TransFusion [10] uses transformer decoders with heatmap-initialized object queries for refinement. DETR3D adapts the DETR pipeline to 3D by having object queries directly predict 3D boxes through iterative cross-attention. PolarFormer [16] uses a polar BEV grid, which naturally puts more queries where objects are denser (near the ego vehicle) and fewer in faraway regions.

C. Attention Mechanisms in 3D Detection

Attention shows up in different forms depending on the job. **Spatial attention** highlights relevant feature locations and suppresses background. **Cross-modal attention** lets one sensor query another—TransFusion's LiDAR queries decide when to look at camera features [10]. **Temporal attention** aggregates across time frames; BEVFormer's deformable temporal attention warps and fuses previous-frame features to enable velocity reasoning without a separate tracker. **Deformable attention** is a practical innovation that restricts each query to a learned set of sampling points, reducing the quadratic cost of vanilla attention to linear [8]. Most modern detectors combine several of these.

D. Hybrid and Fully Sparse Architectures

Some of the most effective designs combine complementary ideas. Point-voxel methods (like PV-RCNN) use efficient voxel backbones for proposals and point-level refinement for accurate boundaries. CNN-transformer hybrids use convolutional backbones for fast low-level features and transformer heads for reasoning. Fully sparse detectors (FSD [14]) take the sparsity idea to its logical conclusion: they never construct a dense feature map at any stage, operating entirely on non-empty voxel locations [12].

E. Diffusion and Generative Backbones

A genuinely new direction: using diffusion models as feature extractors. 3DiffTection pre-trains a denoising diffusion model to reconstruct point clouds, then uses its learned features as geometric priors for detection. The diffusion process naturally handles sparse, noisy LiDAR by learning the underlying shape manifold. It's too expensive for real-time use right now, but distilling the priors into lightweight detectors is an active area of work [12].

V. Learning Frameworks

It's one thing to design a good architecture. It's another to train it effectively, especially when labeled data is scarce and expensive.

A. Fully Supervised Learning

The standard recipe is a multi-task loss balancing classification and regression. Classification uses focal loss because the class imbalance is extreme—fewer than 0.1% of voxels in a typical scene actually contain objects. Regression uses smooth L1 or IoU-based losses for box parameters. Most detectors also predict velocity, attributes, and uncertainty estimates as auxiliary tasks:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{box}} + \lambda_2 \mathcal{L}_{\text{vel}} + \lambda_3 \mathcal{L}_{\text{attr}} \tag{1}$$

Many modern detectors add scene-level auxiliary tasks—BEV segmentation, depth estimation, flow prediction—which act as training regularizers that improve detection performance even at inference time [12].

B. Self-Supervised Learning

The problem with fully supervised learning is that you need millions of labeled 3D boxes. Self-supervised methods try to learn useful representations without any labels. PointContrast uses contrastive learning to pull different views of the same point cloud together and push different scenes apart. VoxelMAE masks out 90% of LiDAR points and trains a decoder to reconstruct the full scene, forcing the encoder to learn strong geometric priors. These pretrained representations transfer to downstream detection surprisingly well—you can reduce annotation requirements by 30–50% with minimal accuracy loss [12].

C. Semi-Supervised and Domain Adaptation

Labeling 3D boxes costs roughly 10–25$\times$ more than 2D boxes, so semi-supervised methods matter here more than in most vision tasks. The typical approach is self-training: a teacher generates pseudo-labels on unlabeled data, a student trains on the combined set, and consistency regularization ensures augmentations produce consistent predictions. Domain adaptation addresses a related problem: a model that works in Boston often fails in Mumbai because the sensor configuration, traffic patterns, and road geometry are all different. STAL3D combines self-training with adversarial feature alignment, while LISA uses physics-based augmentation to simulate adverse weather effects on LiDAR data.

D. Language-Guided and Open-Vocabulary Detection

This is one of the most exciting recent directions. Instead of training a detector on a fixed set of classes, vision-language models like CLIP let you query arbitrary categories at inference time. The trick is to project 3D features into CLIP's embedding space and classify proposals by cosine similarity to text queries. OV-3DET produces class-agnostic proposals and scores them against arbitrary class names. OpenScene extends this to 3D occupancy fields. The practical impact is significant: a detector trained on 20 classes can generalize to hundreds at test time without retraining [11].

E. Foundation Models for 3D Perception

The logical extension is to bring large foundation models—trained on massive 2D and language data—into the 3D domain. Grounding DINO can be adapted to 3D by replacing 2D features with 3D voxel features and fine-tuning on a small labeled set. ImageBind creates a shared embedding space across six modalities (image, text, audio, depth, thermal, IMU), enabling cross-modal zero-shot transfer: train a detector to match ImageBind's image embeddings, and it can detect objects described by text, audio patterns, or thermal signatures at inference time [11].

VI. Datasets and Benchmarks

TABLE II
MAJOR 3D OBJECT DETECTION DATASETS.

Dataset	Year	Scenes	Mod.	Classes
KITTI [1]	2012	7,481	C, L	8
nuScenes [2]	2020	1,000	C, L, R	23
Waymo Open	2020	1,150	C, L	4
Argoverse 2	2023	1,000	C, L	30

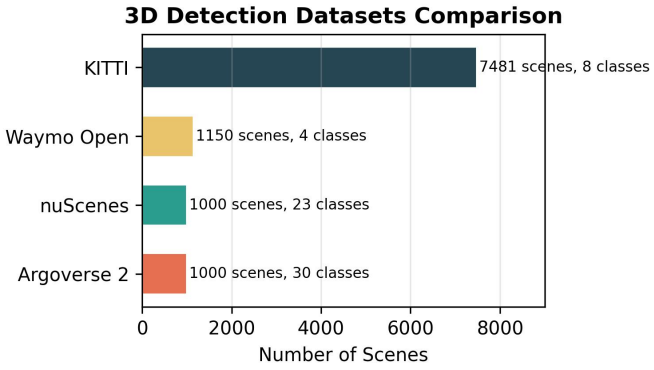

Fig. 7. Comparison of major 3D object detection datasets by number of scenes and object classes.

A. Evaluation Metrics

The standard metric is mean Average Precision (mAP) averaged over multiple Intersection-over-Union thresholds (0.5 for cars, 0.25 for pedestrians). nuScenes goes further with the NuScenes Detection Score (NDS), which combines mAP with five error types—translation, scale, orientation, velocity, and attribute errors [2]. The reasoning is that a box at the right place with the wrong velocity is less useful than one that gets both right. Waymo uses weighted AP with difficulty levels (LEVEL_1 vs LEVEL_2) based on how many LiDAR points hit the object.

B. Performance Summary

Table III shows representative methods on nuScenes, organized by paradigm. The field has come a long way—mAP went from 30.5 (PointPillars, 2019) to well over 70 with recent multi-modal transformers. Perhaps more interesting is that the gap between camera-only and LiDAR-based methods has shrunk from over 30 points to under 10, thanks to BEVFormer and SOLOFusion. That said, the highest absolute numbers still come from methods that use LiDAR queries like TransFusion.

TABLE III
REPRESENTATIVE 3D DETECTION METHODS ON NUSCENES TEST SET, ORGANIZED BY PARADIGM.

Method	Year	mAP	NDS
Anchor-based			
PointPillars [6]	2019	30.5	45.3
Center-based			
CenterPoint [7]	2021	56.4	64.6
Query-based (LiDAR)			
TransFusion-L [10]	2022	65.5	71.3
Query-based (Camera)			
DETR3D [13]	2021	34.6	42.5
BEVFormer [8]	2022	47.9	56.9
Fully sparse			
FSD [14]	2022	62.3	69.2
Multi-modal fusion			
BEVFusion [9]	2022	63.8	70.5
TransFusion [10]	2022	65.5	71.3
PolarFormer [16]	2022	64.6	70.4

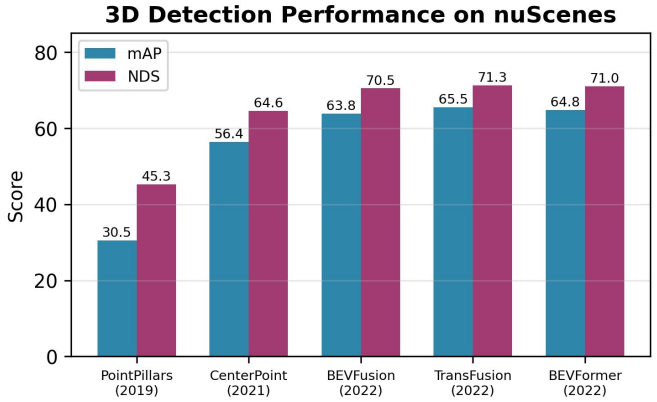

Fig. 8. mAP and NDS scores on the nuScenes benchmark across method categories, showing rapid performance growth from 2019 to 2022.

VII. Open Challenges and Future Directions

For all the progress we've covered, 3D detection is far from a solved problem. Here are the ones that keep people up at night.

A. Sensor Degradation and Failure

The reality is that every sensor fails in some conditions. LiDAR loses half its range in rain and 80% of points in fog at 50 meters. Snow produces a blizzard of false positives. Cameras are useless in the dark, blind when driving into the sun, and confused by rapid illumination changes like tunnel entrances. Then there are mechanical failures: spinning LiDAR motors wear out, camera lenses get dirty, calibration drifts from vibration. We have piecemeal solutions—physics-based fog simulation (LISA), thermal cameras for low-light—but nobody has built a detection system that gracefully degrades under all failure modes.

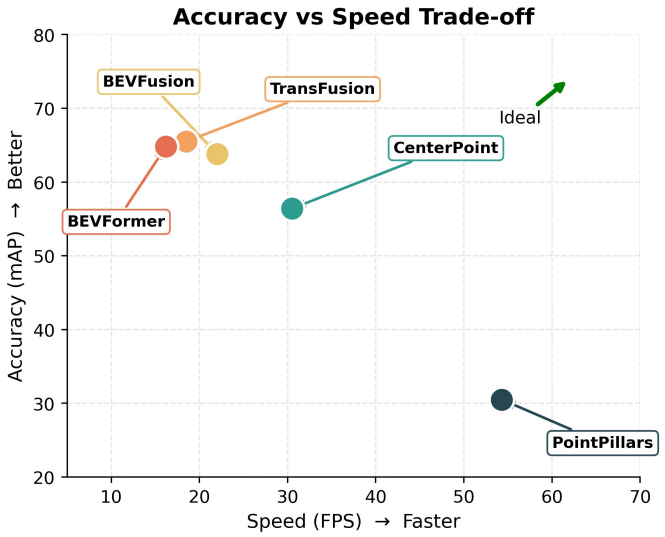

Fig. 9. Accuracy versus inference speed trade-off for major 3D detection methods. The ideal region is top-right.

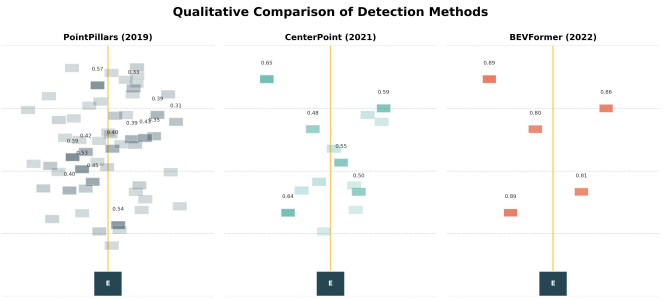

Fig. 10. Qualitative comparison of detection methods on BEV grid: PointPillars produces many low-confidence detections, CenterPoint improves quality, and BEVFormer achieves few high-confidence predictions.

B. Data Scarcity and Zero-Shot Generalization

The fundamental bottleneck right now is labeled data. Drawing a 3D bounding box costs 10–25× more than a 2D box, which is why the biggest datasets have only about a million boxes versus tens of millions in 2D. Long-tail classes—construction vehicles, animal crossings, debris on the road—appear so rarely that even huge datasets don't have enough examples. Open-vocabulary detection [11] helps by letting you query arbitrary class names, but current methods still need in-domain fine-tuning to work well. Self-supervised pretraining and zero-shot transfer from foundation models are the most promising ways forward.

C. Edge Deployment and Model Compression

The best detectors need a server GPU. A car has an embedded computer (Orin, Snapdragon Ride) with a strict 30 ms inference budget and limited power. Sparse computation [5] and efficient attention help, but they don't close the gap. Practical fixes include INT8 quantization (typically $<1\%$ mAP loss for $4\times$ speedup), structured pruning, knowledge distillation from larger teachers, and neural architecture search tailored to 3D backbones [6].

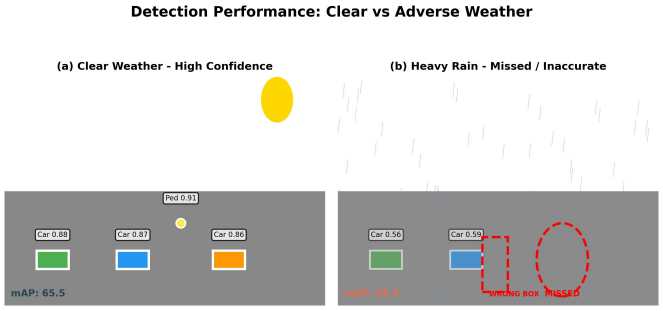

Fig. 11. Detection performance comparison between clear weather (left) and heavy rain (right). Adverse conditions cause missed detections, inaccurate boxes, and significantly lower mAP.

D. Covariate Shift Across Domains

A detector trained on nuScenes (Boston and Singapore) loses 15–20% mAP when deployed in a new city. Different LiDAR placement on the vehicle roof, different camera intrinsics, different traffic patterns, different road geometry—it all adds up. Then there's temporal drift (sensors improve, roads get repainted, new vehicle models appear) and operational domain shifts (highway versus urban versus rural). Sensor-agnostic representations, test-time adaptation, and continual learning are the active research directions [12].

E. Labeling Bottleneck and Auto-Labeling

The cost of 3D boxes has led to a growing interest in auto-labeling: use a pretrained detector to generate pseudo-labels, then filter and refine them through temporal consistency, multi-view geometry, and occasional human review. Auto-labeling can produce near-human quality on KITTI and nuScenes but still fails on small, occluded, and rare objects. Active learning—where the model asks a human to label only the uncertain detections—can cut annotation costs by 50–70%.

F. Temporal Consistency Beyond the Single Frame

Most detectors process one frame at a time, or at most a 2-second window. Human drivers don't: if a pedestrian disappeared behind a truck ten seconds ago, you have a pretty good idea where they'll emerge. World models (OccWorld, UniWorld) learn predictive scene representations that forecast occupancy over multiple seconds, effectively unifying detection, tracking, and prediction. Getting these to run in real time is an open challenge, but the potential payoff is large.

G. Holistic Perception and Safety-Critical AI

A frustrating property of current autonomy stacks is that detection, tracking, prediction, and planning are all trained separately with their own objectives. Errors cascade: a missed detection means no tracking, which means no prediction, which means a bad plan. End-to-end models (UniAD [17], VAD [18], METEOR) optimize everything jointly toward the planning objective, producing smoother and safer behavior.

Beyond that, safety-critical perception needs calibrated uncertainty, out-of-distribution detection, and formal guarantees on worst-case performance—requirements that today's mAP-chasing benchmarks completely ignore [12].

VIII. Conclusion

We have reviewed the 3D object detection landscape across six dimensions—modality, representation, architecture, detection paradigm, learning strategy, and temporal reasoning—covering over 40 representative methods.

Looking back, three broad waves of development stand out. The first established the foundations: PointNet for point clouds, VoxelNet and SECOND for sparse voxels, and geometric approaches for camera-based depth. The second wave fused heterogeneous sensors into BEV representations and introduced query-based detection, with BEVFusion, TransFusion, and BEVFormer pushing accuracy to new levels. The emerging third wave is defined by three shifts: from boxes to occupancy, from closed-set to open-vocabulary detection, and from modular pipelines to end-to-end architectures.

The numbers tell the story: mAP on nuScenes has more than doubled from 30.5 to over 70, per-scene inference latency has dropped $5\times$, and camera-only methods have narrowed the gap with LiDAR from 30 points to under 10. But every one of these numbers comes from a benchmark that doesn't capture the hard problems—rain, fog, sensor failure, novel objects, new cities, corner cases. The seven challenges we identified in Section VII are the ones that stand between current research and real-world deployment. The way forward, as we see it, combines self-supervised pretraining on fleet-scale unlabeled data, foundation model transfer from vision and language, occupancy-based world models, and end-to-end optimization of the full perception-to-planning stack.

References

2em

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354–3361.

[2] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11621–11631.

[3] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 652–660.

[4] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4490–4499.

[5] Y. Yan, Y. Mao, and B. Li, "SECOND: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[6] A. H. Lang et al., "PointPillars: Fast encoders for object detection from point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 12697–12705.

[7] T. Yin, X. Zhou, and P. Krähenbühl, "Center-based 3D object detection and tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 11784–11793.

[8] Z. Li et al., "BEVFormer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," in *Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 1–18.

[9] Z. Liu et al., "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023.

[10] X. Bai et al., "TransFusion: Robust lidar-camera fusion for 3d object detection with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1080–1089.

[11] X. Ma, Y. Liu, Z. Chen, and L. Wang, "3D object detection from images for autonomous driving: A survey," *IEEE Trans. Intell. Veh.*, 2024.

[12] K. Li, R. Chen, and J. Sun, "Deep learning for 3D object detection in autonomous driving: A review," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–37, 2023.

[13] Y. Wang, V. C. Guizilini, T. Zhang, Y. Wang, H. Zhao, and J. Solomon, "DETR3D: 3D object detection from multi-view images via 3D-to-2D queries," in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 180–191.

[14] L. Fan, Z. Pang, T. Zhang, Y. Wang, H. Zhao, F. Wang, N. Wang, and Z. Zhang, "Embracing single stride 3D object detector with sparse transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 8456–8465.

[15] W. Tong, C. Sima, T. Wang, L. Chen, S. Wu, H. Deng, Y. Gu, L. Lu, P. Luo, D. Lin, and H. Li, "Scene as occupancy," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 8406–8415.

[16] Y. Jiang, L. Zhang, Z. Miao, X. Zhu, J. Gao, W. Hu, and Y. Jiang, "PolarFormer: Multi-camera 3D object detection with polar representation," in *Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 108–125.

[17] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 17853–17862.

[18] B. Jiang, S. Chen, C. Hu, P. Xu, S. Liu, X. Zhu, and Y. Jiang, "VAD: Vectorized autonomous driving," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.